

IMPLEMENTASI CODE PREDIKSI CURAH HUJAN AUSTRALIA (RANDOM FOREST CLASSIFIER)

Link Source Code: [Implementasi_Code_Tubes_DM_Kelompok_1_Missing_Value](#)

IMPORT LIBRARY

```
# Library dasar manipulasi dan komputasi data
import numpy as np
import pandas as pd
import time
import warnings
warnings.filterwarnings('ignore')

# Library visualisasi
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import missingno as msno # Visualisasi pola missing value

# Library preprocessing
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# Library imputasi
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer # Wajib
sebelum import IterativeImputer
from sklearn.impute import IterativeImputer

# Library model dan evaluasi
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix
)

# Pengaturan tampilan
pd.set_option('display.max_columns', 30)
```

```
pd.set_option('display.float_format', '{:.4f}'.format)
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams['figure.dpi'] = 100

print("☑ Seluruh library berhasil diimport.")
```

LOAD DATASET

```
# Load dataset dari file CSV (Link GitHub Raw)
# Pastikan ada https:// di depannya ya!
url_github = 'https://raw.githubusercontent.com/GarsRayy/Dataset-Data-
mining/refs/heads/main/weatherAUS.csv'

# Simpan ke variabel df_raw agar cocok dengan print di bawahnya
df_raw = pd.read_csv(url_github)

print(f"\nDimensi Dataset: {df_raw.shape[0]:,} baris ×
{df_raw.shape[1]} kolom")
print(f"\nKolom-kolom dalam dataset:")
print(list(df_raw.columns))

print(f"\n--- 5 Baris Pertama Dataset ---")
df_raw.head()

☑ Seluruh library berhasil diimport.

Dimensi Dataset: 145,460 baris × 23 kolom

Kolom-kolom dalam dataset:
['Date', 'Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation',
'Sunshine', 'WindGustDir', 'WindGustSpeed', 'WindDir9am',
'WindDir3pm', 'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am',
'Humidity3pm', 'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Cloud3pm',
'Temp9am', 'Temp3pm', 'RainToday', 'RainTomorrow']

--- 5 Baris Pertama Dataset ---

{"type": "dataframe", "variable_name": "df_raw"}
```

EDA UMUM

```
print("=" * 60)
print("INFORMASI TIPE DATA & NILAI KOSONG")
print("=" * 60)
df_raw.info()

print("\n" + "=" * 60)
print("STATISTIK DESKRIPTIF FITUR NUMERIK")
print("=" * 60)
display(df_raw.describe())
```

```

# =====
# ANALISIS MISSING VALUE SECARA KUANTITATIF
# =====

print("\n" + "=" * 60)
print("RINGKASAN MISSING VALUE PER KOLOM")
print("=" * 60)

# Hitung jumlah dan persentase missing value untuk setiap kolom
missing_count = df_raw.isnull().sum()
missing_pct = (missing_count / len(df_raw) * 100).round(2)
missing_df = pd.DataFrame({
    'Jumlah Missing': missing_count,
    'Persentase (%)': missing_pct
}).sort_values('Persentase (%)', ascending=False)

# Tampilkan hanya kolom yang memiliki nilai kosong
print(missing_df[missing_df['Jumlah Missing'] > 0].to_string())
print(f"\nTotal kolom dengan missing value: {(missing_count > 0).sum()} dari {len(df_raw.columns)} kolom")

# =====
# VISUALISASI 1: DISTRIBUSI TARGET 'RainTomorrow'
# =====

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel kiri: Bar chart distribusi kelas
target_counts = df_raw['RainTomorrow'].value_counts()
colors = ['#3498db', '#e74c3c']
bars = axes[0].bar(target_counts.index, target_counts.values,
color=colors, edgecolor='black', linewidth=0.8)
axes[0].set_title('Distribusi Kelas Target: RainTomorrow',
    fontsize=14, fontweight='bold')
axes[0].set_xlabel('Kelas', fontsize=12)
axes[0].set_ylabel('Jumlah Observasi', fontsize=12)
for bar, count in zip(bars, target_counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2., bar.get_height() +
500,
                f'{count:,}\n'
n(({count/len(df_raw.dropna(subset=["RainTomorrow"]))*100:.1f}%)',
                ha='center', va='bottom', fontsize=11,
fontweight='bold')

# Panel kanan: Pie chart proporsi
axes[1].pie(target_counts.values, labels=target_counts.index,
autopct='%1.1f%%',
            colors=colors, startangle=90, textprops={'fontsize': 12})
axes[1].set_title('Proporsi Kelas Target: RainTomorrow', fontsize=14,

```

```

fontweight='bold')

plt.suptitle('Analisis Distribusi Variabel Target', fontsize=15,
fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('fig1_target_distribution.png', bbox_inches='tight',
dpi=100)
plt.show()
print("☐ Gambar 1: Distribusi Target berhasil dibuat.")

# =====
# VISUALISASI 2: POLA MISSING VALUE (MISSINGNO)
# =====

print("\n" + "=" * 60)
print("VISUALISASI POLA MISSING VALUE")
print("=" * 60)

# Bar chart proporsi missing value menggunakan missingno
fig, axes = plt.subplots(1, 2, figsize=(18, 6))

# Plot 1: Bar chart missing value
msno.bar(df_raw, figsize=(9, 5), fontsize=10, ax=axes[0],
color='#2980b9')
axes[0].set_title('Gambar 2a: Proporsi Data Tersedia per Kolom\
n(Missingno Bar Chart)',
                fontsize=13, fontweight='bold')

# Plot 2: Matrix visualisasi pola missing
msno.matrix(df_raw, figsize=(9, 5), fontsize=8, ax=axes[1],
color=(0.15, 0.47, 0.72))
axes[1].set_title('Gambar 2b: Pola Missing Value (Matriks)\n(Putih =
Data Hilang)',
                fontsize=13, fontweight='bold')

plt.tight_layout()
plt.savefig('fig2_missing_pattern.png', bbox_inches='tight', dpi=100)
plt.show()

# Heatmap korelasi antar missing value (untuk mendeteksi MAR)
plt.figure(figsize=(12, 8))
msno.heatmap(df_raw, fontsize=10, cmap='RdYlGn')
plt.title('Gambar 3: Heatmap Korelasi Antar Missing Value\n(Nilai
tinggi = Missing bersamaan)',
        fontsize=13, fontweight='bold')
plt.savefig('fig3_missing_heatmap.png', bbox_inches='tight', dpi=100)
plt.show()
print("☐ Visualisasi pola missing value berhasil dibuat.")

```

INFORMASI TIPE DATA & NILAI KOSONG

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145460 entries, 0 to 145459
Data columns (total 23 columns):

#	Column	Non-Null Count	Dtype
0	Date	145460 non-null	object
1	Location	145460 non-null	object
2	MinTemp	143975 non-null	float64
3	MaxTemp	144199 non-null	float64
4	Rainfall	142199 non-null	float64
5	Evaporation	82670 non-null	float64
6	Sunshine	75625 non-null	float64
7	WindGustDir	135134 non-null	object
8	WindGustSpeed	135197 non-null	float64
9	WindDir9am	134894 non-null	object
10	WindDir3pm	141232 non-null	object
11	WindSpeed9am	143693 non-null	float64
12	WindSpeed3pm	142398 non-null	float64
13	Humidity9am	142806 non-null	float64
14	Humidity3pm	140953 non-null	float64
15	Pressure9am	130395 non-null	float64
16	Pressure3pm	130432 non-null	float64
17	Cloud9am	89572 non-null	float64
18	Cloud3pm	86102 non-null	float64
19	Temp9am	143693 non-null	float64
20	Temp3pm	141851 non-null	float64
21	RainToday	142199 non-null	object
22	RainTomorrow	142193 non-null	object

dtypes: float64(16), object(7)

memory usage: 25.5+ MB

STATISTIK DESKRIPTIF FITUR NUMERIK

```
{"summary":{"name": "print(\\u2705 Visualisasi pola missing value berhasil dibuat",\n  "rows": 8,\n  "fields": [\n    {\n      "column": "MinTemp",\n      "properties": {\n        "dtype": "number",\n        "std": 50898.7852892957,\n        "min": -8.5,\n        "max": 143975.0,\n        "num_unique_values": 8,\n        "samples": [\n          12.19403438096892,\n          12.0,\n          143975.0\n        ],\n        "semantic_type": "",\n        "description": ""\n      }\n    },\n    {\n      "column": "MaxTemp",\n      "properties": {\n        "dtype": "number",\n        "std": 50974.85841852006,\n        "min": -4.8,\n        "max": 144199.0,\n        "num_unique_values": 8,\n        "samples": [\n
```

```
23.22134827564685,\n                22.6,\n                144199.0\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n            },\n            {\n                \"column\": \"Rainfall\",\n                \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 50255.778414581844,\n                    \"min\": 0.0,\n                    \"max\": 142199.0,\n                    \"num_unique_values\": 6,\n                    \"samples\": [\n                        142199.0,\n                        2.3609181499166656,\n                        371.0\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\",\n                },\n                {\n                    \"column\": \"Evaporation\",\n                    \"properties\": {\n                        \"dtype\": \"number\",\n                        \"std\": 29219.741370533065,\n                        \"min\": 0.0,\n                        \"max\": 82670.0,\n                        \"num_unique_values\": 8,\n                        \"samples\": [\n                            5.468231522922462,\n                            4.8,\n                            82670.0\n                        ],\n                        \"semantic_type\": \"\",\n                        \"description\": \"\",\n                    },\n                    {\n                        \"column\": \"Sunshine\",\n                        \"properties\": {\n                            \"dtype\": \"number\",\n                            \"std\": 26734.965465626912,\n                            \"min\": 0.0,\n                            \"max\": 75625.0,\n                            \"num_unique_values\": 8,\n                            \"samples\": [\n                                7.6111775206611565,\n                                8.4,\n                                75625.0\n                            ],\n                            \"semantic_type\": \"\",\n                            \"description\": \"\",\n                        },\n                        {\n                            \"column\": \"WindGustSpeed\",\n                            \"properties\": {\n                                \"dtype\": \"number\",\n                                \"std\": 47783.58317859071,\n                                \"min\": 6.0,\n                                \"max\": 135197.0,\n                                \"num_unique_values\": 8,\n                                \"samples\": [\n                                    40.03523007167319,\n                                    39.0,\n                                    135197.0\n                                ],\n                                \"semantic_type\": \"\",\n                                \"description\": \"\",\n                            },\n                            {\n                                \"column\": \"WindSpeed9am\",\n                                \"properties\": {\n                                    \"dtype\": \"number\",\n                                    \"std\": 50793.46953742816,\n                                    \"min\": 0.0,\n                                    \"max\": 143693.0,\n                                    \"num_unique_values\": 8,\n                                    \"samples\": [\n                                        14.043425914971502,\n                                        13.0,\n                                        143693.0\n                                    ],\n                                    \"semantic_type\": \"\",\n                                    \"description\": \"\",\n                                },\n                                {\n                                    \"column\": \"WindSpeed3pm\",\n                                    \"properties\": {\n                                        \"dtype\": \"number\",\n                                        \"std\": 50336.6925776163,\n                                        \"min\": 0.0,\n                                        \"max\": 142398.0,\n                                        \"num_unique_values\": 8,\n                                        \"samples\": [\n                                            18.662656778887342,\n                                            19.0,\n                                            142398.0\n                                        ],\n                                        \"semantic_type\": \"\",\n                                        \"description\": \"\",\n                                    },\n                                    {\n                                        \"column\": \"Humidity9am\",\n                                        \"properties\": {\n                                            \"dtype\": \"number\",\n                                            \"std\": 50469.45865841999,\n                                            \"min\": 0.0,\n                                            \"max\": 142806.0,\n                                            \"num_unique_values\": 8,\n                                            \"samples\": [\n                                                68.88083133761887,\n                                                70.0,\n                                                142806.0\n                                            ],\n                                            \"semantic_type\": \"\",\n                                            \"description\": \"\",\n                                        },\n                                        {\n                                            \"column\": \"Humidity3pm\",\n                                            \"properties\": {\n                                                \"dtype\": \"number\",\n                                                \"std\": 49817.88705434561,\n                                                \"min\": 0.0,\n                                                \"max\": 140953.0,\n                                                \"num_unique_values\": 8,\n                                                \"samples\": [\n                                                    51.5391158755046,\n                                                    52.0,\n                                                    140953.0\n                                                ],\n                                                \"semantic_type\": \"\",\n                                                \"description\": \"\",\n                                            },\n                                        }\n                                    }\n                                }\n                            }\n                        }\n                    }\n                }\n            }\n        ]\n    }\n}
```

```

{"semantic_type": "\"",\n
  },\n
  {\n
    \"column\": \"Pressure9am\", \n
    \"properties\": {\n
      \"dtype\": \"number\", \n
      \"std\": 45794.902527110266, \n
      \"min\": 7.106530287523588, \n
      \"max\": 130395.0, \n
      \"num_unique_values\": 8, \n
      \"samples\": [\n
        1017.6499397983052, \n
        1017.6, \n
        130395.0\n
      ], \n
      \"semantic_type\": \"\", \n
      \"description\": \"\" \n
    }, \n
    {\n
      \"column\": \"Pressure3pm\", \n
      \"properties\": {\n
        \"dtype\": \"number\", \n
        \"std\": 45808.7130186032, \n
        \"min\": 7.0374138081677975, \n
        \"max\": 130432.0, \n
        \"num_unique_values\": 8, \n
        \"samples\": [\n
          1015.2558888309618, \n
          1015.2, \n
          130432.0\n
        ], \n
        \"semantic_type\": \"\", \n
        \"description\": \"\" \n
      }, \n
      {\n
        \"column\": \"Cloud9am\", \n
        \"properties\": {\n
          \"dtype\": \"number\", \n
          \"std\": 31667.00281817111, \n
          \"min\": 0.0, \n
          \"max\": 89572.0, \n
          \"num_unique_values\": 8, \n
          \"samples\": [\n
            4.4474612602152455, \n
            5.0, \n
            89572.0\n
          ], \n
          \"semantic_type\": \"\", \n
          \"description\": \"\" \n
        }, \n
        {\n
          \"column\": \"Cloud3pm\", \n
          \"properties\": {\n
            \"dtype\": \"number\", \n
            \"std\": 30440.12730861951, \n
            \"min\": 0.0, \n
            \"max\": 86102.0, \n
            \"num_unique_values\": 8, \n
            \"samples\": [\n
              4.509930082924903, \n
              5.0, \n
              86102.0\n
            ], \n
            \"semantic_type\": \"\", \n
            \"description\": \"\" \n
          }, \n
          {\n
            \"column\": \"Temp9am\", \n
            \"properties\": {\n
              \"dtype\": \"number\", \n
              \"std\": 50797.74079696554, \n
              \"min\": -7.2, \n
              \"max\": 143693.0, \n
              \"num_unique_values\": 8, \n
              \"samples\": [\n
                16.990631415587398, \n
                16.7, \n
                143693.0\n
              ], \n
              \"semantic_type\": \"\", \n
              \"description\": \"\" \n
            }, \n
            {\n
              \"column\": \"Temp3pm\", \n
              \"properties\": {\n
                \"dtype\": \"number\", \n
                \"std\": 50145.135225618586, \n
                \"min\": -5.4, \n
                \"max\": 141851.0, \n
                \"num_unique_values\": 8, \n
                \"samples\": [\n
                  21.68339031800974, \n
                  21.1, \n
                  141851.0\n
                ], \n
                \"semantic_type\": \"\", \n
                \"description\": \"\" \n
              }, \n
              {\n
            } \n
          } \n
        } \n
      ], \n
      \"type\": \"dataframe\"}

```

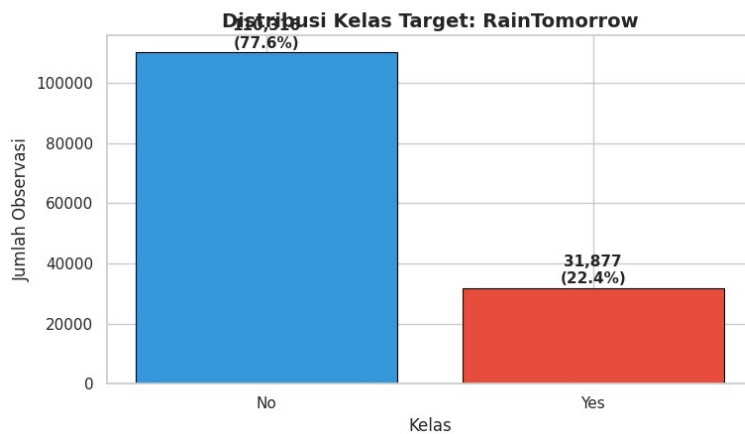
RINGKASAN MISSING VALUE PER KOLOM

	Jumlah Missing	Persentase (%)
Sunshine	69835	48.0100
Evaporation	62790	43.1700
Cloud3pm	59358	40.8100
Cloud9am	55888	38.4200
Pressure9am	15065	10.3600

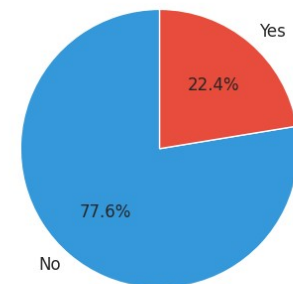
Pressure3pm	15028	10.3300
WindDir9am	10566	7.2600
WindGustDir	10326	7.1000
WindGustSpeed	10263	7.0600
Humidity3pm	4507	3.1000
WindDir3pm	4228	2.9100
Temp3pm	3609	2.4800
RainTomorrow	3267	2.2500
Rainfall	3261	2.2400
RainToday	3261	2.2400
WindSpeed3pm	3062	2.1100
Humidity9am	2654	1.8200
WindSpeed9am	1767	1.2100
Temp9am	1767	1.2100
MinTemp	1485	1.0200
MaxTemp	1261	0.8700

Total kolom dengan missing value: 21 dari 23 kolom

Analisis Distribusi Variabel Target



Proporsi Kelas Target: RainTomorrow

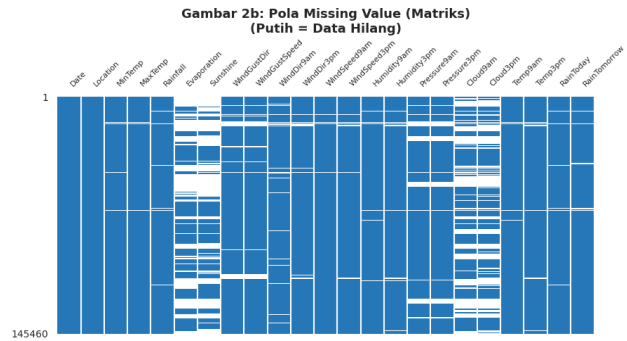
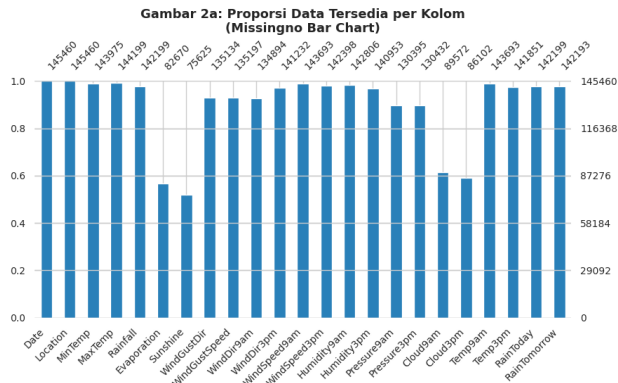


□ Gambar 1: Distribusi Target berhasil dibuat.

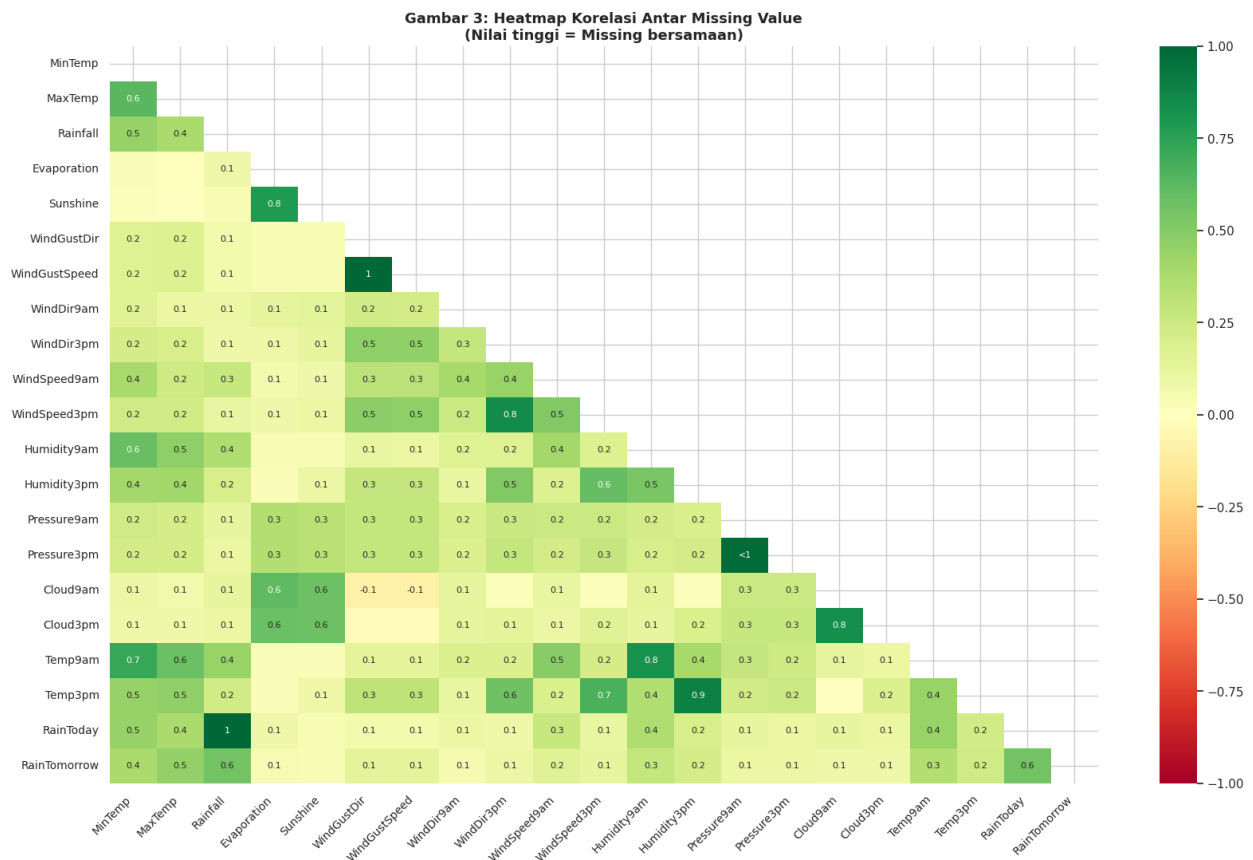
=====

VISUALISASI POLA MISSING VALUE

=====



<Figure size 1200x800 with 0 Axes>



☐ Visualisasi pola missing value berhasil dibuat.

PREPROCESSING: ENCODING FITUR KATEGORIK

```
df = df_raw.copy()
df = df.drop(columns=['Date'])
```

--- FIX: Hapus baris di mana TARGET bernilai NaN SEBELUM encoding

```

---
# Ini mencegah 'nan' ikut ter-encode sebagai kelas ketiga
df = df[df['RainTomorrow'].notna()].copy()
print(f"Baris setelah filter target NaN: {len(df):,}")

# Kolom kategorik selain target
categorical_cols =
df.select_dtypes(include=['object']).columns.tolist()
categorical_cols_no_target = [c for c in categorical_cols if c !=
'RainTomorrow']

le = LabelEncoder()

# Encode fitur kategorik (bukan target) – NaN menjadi 'nan' string,
itu OK untuk fitur
for col in categorical_cols_no_target:
    df[col] = df[col].astype(str)
    df[col] = le.fit_transform(df[col])

# Encode TARGET secara eksplisit: 'No' → 0, 'Yes' → 1
# Map manual lebih aman daripada LabelEncoder untuk target
df['RainTomorrow'] = df['RainTomorrow'].map({'No': 0, 'Yes': 1})

print(f"\nNilai unik RainTomorrow:
{sorted(df['RainTomorrow'].unique())}")
print(f"(Harus hanya [0, 1])")

X = df.drop(columns=['RainTomorrow'])
y = df['RainTomorrow']

print(f"\nDimensi X: {X.shape} | Dimensi y: {y.shape}")
print(f"Distribusi kelas:\n{y.value_counts()}")

Baris setelah filter target NaN: 142,193

Nilai unik RainTomorrow: [np.int64(0), np.int64(1)]
(Harus hanya [0, 1])

Dimensi X: (142193, 21) | Dimensi y: (142193,)
Distribusi kelas:
RainTomorrow
0      110316
1       31877
Name: count, dtype: int64

```

TRAIN-TEST SPLIT (80:20)

```

# Hapus baris di mana target (y) bernilai NaN sebelum split
# (karena kita tidak bisa melatih model dengan target yang tidak
diketahui)

```

```

valid_idx = y.notna()
X_valid = X[valid_idx].copy()
y_valid = y[valid_idx].copy()

print(f"Jumlah sampel valid (target tidak NaN): {len(X_valid):,}")

# Split data: 80% latih, 20% uji, stratified untuk mempertahankan
proporsi kelas
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_valid, y_valid,
    test_size=0.2,
    random_state=42,
    stratify=y_valid # Pastikan proporsi kelas seimbang di train dan
test
)

print(f"\nTrain-Test Split berhasil dilakukan!")
print(f"    └─ X_train_raw: {X_train_raw.shape} | y_train:
{y_train.shape}")
print(f"    └─ X_test_raw:  {X_test_raw.shape} | y_test:
{y_test.shape}")
print(f"\nDistribusi kelas di TRAIN set:")
print(y_train.value_counts(normalize=True).round(4))
print(f"\nDistribusi kelas di TEST set:")
print(y_test.value_counts(normalize=True).round(4))

print("\n" + "="*60)
print("    CATATAN PENTING: Imputer akan di-FIT hanya pada
X_train_raw")
print("    dan kemudian di-TRANSFORM pada X_train_raw dan X_test_raw.")
print("    Ini memastikan ZERO DATA LEAKAGE dari test set ke train
set.")
print("="*60)

```

Jumlah sampel valid (target tidak NaN): 142,193

Train-Test Split berhasil dilakukan!

```

└─ X_train_raw: (113754, 21) | y_train: (113754,)
└─ X_test_raw:  (28439, 21) | y_test:  (28439,)

```

Distribusi kelas di TRAIN set:

RainTomorrow

0 0.7758

1 0.2242

Name: proportion, dtype: float64

Distribusi kelas di TEST set:

RainTomorrow

0 0.7758

1 0.2242

Name: proportion, dtype: float64

```
=====
CATATAN PENTING: Imputer akan di-FIT hanya pada X_train_raw
dan kemudian di-TRANSFORM pada X_train_raw dan X_test_raw.
Ini memastikan ZERO DATA LEAKAGE dari test set ke train set.
=====
```

EKSEKUSI 5 METODE IMPUTASI MISSING VALUE

```
# Prinsip: SEMUA imputer hanya di-fit pada X_train_raw!
print("=" * 65)
print("  PIPELINE IMPUTASI MISSING VALUE (5 METODE)")
print("=" * 65)

# --- METODE 0: BASELINE (Drop NA) ---
# Strategi paling sederhana: hapus semua baris yang mengandung missing
# value.
# Ini dilakukan secara terpisah pada train dan test.
print("\n[0/4] Metode Baseline (Drop NA)...")

X_train_base = X_train_raw.dropna()
y_train_base = y_train[X_train_base.index] # Filter target sesuai
# indeks yang tersisa
X_test_base = X_test_raw.dropna()
y_test_base = y_test[X_test_base.index]

print(f"  Ukuran X_train setelah dropna: {X_train_base.shape} "
      f"(hilang {len(X_train_raw) - len(X_train_base):,} baris, "
      f"{(1 - len(X_train_base)/len(X_train_raw))*100:.1f}% data)")
print(f"  Ukuran X_test setelah dropna: {X_test_base.shape}")

# --- METODE 1: MEAN IMPUTATION ---
# Isi nilai kosong dengan rata-rata (mean) setiap kolom dari training
# set.
print("\n[1/4] Metode Mean Imputation...")
t_start = time.time()

imputer_mean = SimpleImputer(strategy='mean') # Hitung mean dari
# X_train
imputer_mean.fit(X_train_raw) # FIT HANYA pada data latih!

X_train_mean = pd.DataFrame(
    imputer_mean.transform(X_train_raw),
    columns=X_train_raw.columns
)
X_test_mean = pd.DataFrame(
    imputer_mean.transform(X_test_raw), # Transform menggunakan
# statistik dari train
```

```

        columns=X_test_raw.columns
    )

    print(f"    Selesai dalam {time.time() - t_start:.2f} detik.")
    print(f"    Sisa missing value di X_train:
    {X_train_mean.isnull().sum().sum()}")

# --- METODE 2: MEDIAN IMPUTATION ---
# Isi nilai kosong dengan median setiap kolom dari training set.
# Lebih robust terhadap outlier dibandingkan mean.
print("\n[2/4] Metode Median Imputation...")
t_start = time.time()

imputer_median = SimpleImputer(strategy='median') # Hitung median
dari X_train
imputer_median.fit(X_train_raw) # FIT HANYA pada data latih!

X_train_median = pd.DataFrame(
    imputer_median.transform(X_train_raw),
    columns=X_train_raw.columns
)
X_test_median = pd.DataFrame(
    imputer_median.transform(X_test_raw),
    columns=X_test_raw.columns
)

print(f"    Selesai dalam {time.time() - t_start:.2f} detik.")
print(f"    Sisa missing value di X_train:
    {X_train_median.isnull().sum().sum()}")

# --- METODE 3: KNN IMPUTER (k=5) ---
# Isi nilai kosong menggunakan rata-rata tertimbang dari 5 tetangga
terdekat.
# Mempertimbangkan relasi antar variabel secara implisit.
print("\n[3/4] Metode KNN Imputer (k=5)... (proses lebih lama)")
t_start = time.time()

imputer_knn = KNNImputer(n_neighbors=5, weights='uniform')
imputer_knn.fit(X_train_raw) # FIT HANYA pada data latih!

X_train_knn = pd.DataFrame(
    imputer_knn.transform(X_train_raw),
    columns=X_train_raw.columns
)
X_test_knn = pd.DataFrame(
    imputer_knn.transform(X_test_raw),
    columns=X_test_raw.columns
)

```

```

t_knn = time.time() - t_start
print(f"    Selesai dalam {t_knn:.2f} detik.")
print(f"    Sisa missing value di X_train:
{X_train_knn.isnull().sum().sum()}")

# --- METODE 4: MICE / IterativeImputer ---
# Imputasi multivariat berantai (MICE). Setiap fitur dimodelkan secara
iteratif
# menggunakan fitur-fitur lain sebagai prediktor.
print("\n[4/4] Metode MICE (IterativeImputer)... (proses paling
lama)")
t_start = time.time()

imputer_mice = IterativeImputer(
    max_iter=10,          # Jumlah iterasi imputasi berantai
    random_state=42,      # Untuk reproduibilitas
    initial_strategy='mean' # Inisialisasi awal menggunakan mean
)
imputer_mice.fit(X_train_raw) # FIT HANYA pada data latih!

X_train_mice = pd.DataFrame(
    imputer_mice.transform(X_train_raw),
    columns=X_train_raw.columns
)
X_test_mice = pd.DataFrame(
    imputer_mice.transform(X_test_raw),
    columns=X_test_raw.columns
)

t_mice = time.time() - t_start
print(f"    Selesai dalam {t_mice:.2f} detik.")
print(f"    Sisa missing value di X_train:
{X_train_mice.isnull().sum().sum()}")

print("\n" + "=" * 65)
print(" Semua metode imputasi selesai dieksekusi!")
print("=" * 65)

```

```

=====
PIPELINE IMPUTASI MISSING VALUE (5 METODE)
=====

[0/4] Metode Baseline (Drop NA)...
    Ukuran X_train setelah dropna: (46466, 21) (hilang 67,288 baris,
59.2% data)
    Ukuran X_test setelah dropna: (11624, 21)

[1/4] Metode Mean Imputation...

```

```

Selesai dalam 0.18 detik.
Sisa missing value di X_train: 0

[2/4] Metode Median Imputation...
Selesai dalam 0.74 detik.
Sisa missing value di X_train: 0

[3/4] Metode KNN Imputer (k=5)... (proses lebih lama)
Selesai dalam 1371.38 detik.
Sisa missing value di X_train: 0

[4/4] Metode MICE (IterativeImputer)... (proses paling lama)
Selesai dalam 59.35 detik.
Sisa missing value di X_train: 0

=====
Semua metode imputasi selesai dieksekusi!
=====

```

VISUALISASI: BEFORE vs AFTER IMPUTASI

```

# Kolom yang divisualisasikan: Humidity3pm (missing cukup signifikan)
col_viz = 'Humidity3pm' # Pilih kolom dengan missing value yang cukup signifikan

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle(
    f'Gambar 4: Distribusi Kolom "{col_viz}" - Before vs After Imputasi\n'
    f'(Garis merah putus-putus = nilai mean dari data original)',
    fontsize=14, fontweight='bold', y=1.01
)

# Hitung mean dan median dari data original (sebelum imputasi) untuk referensi
original_mean = X_train_raw[col_viz].mean()
original_median = X_train_raw[col_viz].median()

# --- Subplot 1: Data BEFORE (Original, dengan Missing) ---
ax = axes[0, 0]
X_train_raw[col_viz].dropna().hist(bins=40, ax=ax, color='#7f8c8d',
edgecolor='white', alpha=0.8)
ax.axvline(original_mean, color='red', linestyle='--', linewidth=1.5,
label=f'Mean = {original_mean:.2f}')
ax.set_title(f'BEFORE Imputasi\n(Missing: {X_train_raw[col_viz].isna().sum():,} nilai)', fontsize=11,
fontweight='bold')
ax.set_xlabel(col_viz)
ax.set_ylabel('Frekuensi')
ax.legend(fontsize=9)

```

```

# --- Subplot 2: AFTER Mean Imputation ---
ax = axes[0, 1]
X_train_mean[col_viz].hist(bins=40, ax=ax, color='#3498db',
edgecolor='white', alpha=0.8)
ax.axvline(original_mean, color='red', linestyle='--', linewidth=1.5)
ax.set_title('AFTER Mean Imputation\n(Spike pada nilai mean)',
fontsize=11, fontweight='bold')
ax.set_xlabel(col_viz)
ax.set_ylabel('Frekuensi')

# --- Subplot 3: AFTER Median Imputation ---
ax = axes[0, 2]
X_train_median[col_viz].hist(bins=40, ax=ax, color='#2ecc71',
edgecolor='white', alpha=0.8)
ax.axvline(original_median, color='red', linestyle='--',
linewidth=1.5, label=f'Median = {original_median:.2f}')
ax.set_title('AFTER Median Imputation\n(Spike pada nilai median)',
fontsize=11, fontweight='bold')
ax.set_xlabel(col_viz)
ax.set_ylabel('Frekuensi')
ax.legend(fontsize=9)

# --- Subplot 4: AFTER KNN Imputation ---
ax = axes[1, 0]
X_train_knn[col_viz].hist(bins=40, ax=ax, color='#9b59b6',
edgecolor='white', alpha=0.8)
ax.axvline(original_mean, color='red', linestyle='--', linewidth=1.5)
ax.set_title('AFTER KNN Imputation (k=5)\n(Distribusi lebih
preserved)', fontsize=11, fontweight='bold')
ax.set_xlabel(col_viz)
ax.set_ylabel('Frekuensi')

# --- Subplot 5: AFTER MICE Imputation ---
ax = axes[1, 1]
X_train_mice[col_viz].hist(bins=40, ax=ax, color='#e67e22',
edgecolor='white', alpha=0.8)
ax.axvline(original_mean, color='red', linestyle='--', linewidth=1.5)
ax.set_title('AFTER MICE Imputation\n(Distribusi paling mendekati
asli)', fontsize=11, fontweight='bold')
ax.set_xlabel(col_viz)
ax.set_ylabel('Frekuensi')

# --- Subplot 6: Perbandingan statistik deskriptif ---
ax = axes[1, 2]
ax.axis('off')

# Buat tabel perbandingan statistik
stats_data = {
    'Metode': ['Before\n(Original)', 'Mean\nImputation', 'Median\n

```



```

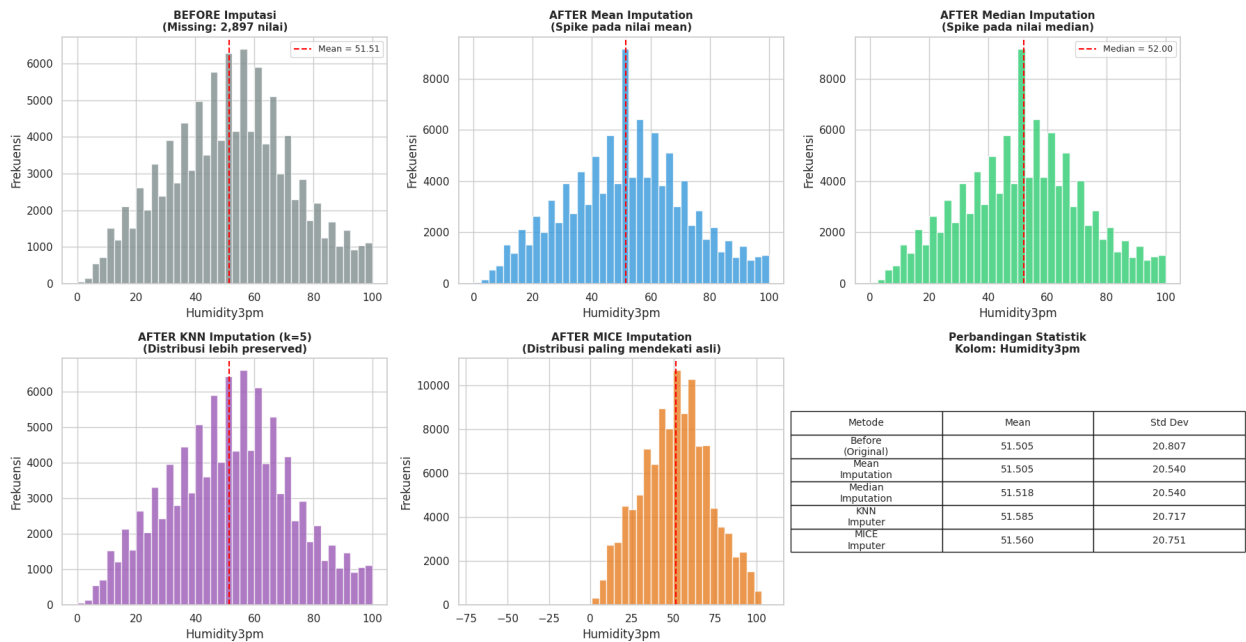
nImputation', 'KNN\nImputer', 'MICE\nImputer'],
    'Mean': [
        X_train_raw[col_viz].mean(),
        X_train_mean[col_viz].mean(),
        X_train_median[col_viz].mean(),
        X_train_knn[col_viz].mean(),
        X_train_mice[col_viz].mean()
    ],
    'Std': [
        X_train_raw[col_viz].std(),
        X_train_mean[col_viz].std(),
        X_train_median[col_viz].std(),
        X_train_knn[col_viz].std(),
        X_train_mice[col_viz].std()
    ]
}

table_data = [[m, f'{mn:.3f}', f'{s:.3f}']
               for m, mn, s in zip(stats_data['Metode'],
stats_data['Mean'], stats_data['Std'])]
table = ax.table(
    cellText=table_data,
    colLabels=['Metode', 'Mean', 'Std Dev'],
    loc='center', cellLoc='center'
)
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1.4, 2)
ax.set_title(f'Perbandingan Statistik\nKolom: {col_viz}', fontsize=11,
fontweight='bold')

plt.tight_layout()
plt.savefig('fig4_before_after_imputation.png', bbox_inches='tight',
dpi=100)
plt.show()
print("❏ Gambar 4: Visualisasi Before vs After berhasil dibuat.")

```

Gambar 4: Distribusi Kolom "Humidity3pm" — Before vs After Imputasi
(Garis merah putus-putus = nilai mean dari data original)



□ Gambar 4: Visualisasi Before vs After berhasil dibuat.

TRAINING RANDOM FOREST & EVALUASI SEMUA SKENARIO

```
print("=" * 65)
print("  TRAINING RANDOM FOREST CLASSIFIER (5 SKENARIO)")
print("=" * 65)

scenarios = [
    {
        'name_short': 'Baseline',
        'X_train': X_train_base, 'X_test': X_test_base,
        'y_train': y_train_base, 'y_test': y_test_base
    },
    {
        'name_short': 'Mean',
        'X_train': X_train_mean, 'X_test': X_test_mean,
        'y_train': y_train,      'y_test': y_test
    },
    {
        'name_short': 'Median',
        'X_train': X_train_median, 'X_test': X_test_median,
        'y_train': y_train,      'y_test': y_test
    },
    {
        'name_short': 'KNN',
```

```

        'X_train': X_train_knn, 'X_test': X_test_knn,
        'y_train': y_train,    'y_test': y_test
    },
    {
        'name_short': 'MICE',
        'X_train': X_train_mice, 'X_test': X_test_mice,
        'y_train': y_train,    'y_test': y_test
    },
]

results = []

for i, sc in enumerate(scenarios):
    print(f"\n[{i+1}/5] Training dengan metode:
    {sc['name_short']}...")

    # Catat waktu mulai di LUAR blok apa pun
    t_start = time.time()

    rf = RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1)
    rf.fit(sc['X_train'], sc['y_train'])

    y_pred = rf.predict(sc['X_test'])
    y_prob_all = rf.predict_proba(sc['X_test'])

    acc = accuracy_score(sc['y_test'], y_pred)
    prec = precision_score(sc['y_test'], y_pred, average='weighted',
zero_division=0)
    rec = recall_score(sc['y_test'], y_pred, average='weighted',
zero_division=0)
    f1 = f1_score(sc['y_test'], y_pred, average='weighted',
zero_division=0)

    # AUC-ROC: deteksi otomatis binary vs multiclass
    n_classes = len(rf.classes_)
    if n_classes == 2:
        auc = roc_auc_score(sc['y_test'], y_prob_all[:, 1])
    else:
        auc = roc_auc_score(sc['y_test'], y_prob_all,
                            multi_class='ovr', average='weighted')

    # Hitung waktu SETELAH semua proses selesai
    t_elapsed = time.time() - t_start

    results.append({
        'Metode': sc['name_short'],
        'N Train': len(sc['X_train']),
        'N Test': len(sc['X_test']),
        'Accuracy': acc,

```

```

        'Precision': prec,
        'Recall':    rec,
        'F1-Score':  f1,
        'AUC-ROC':   auc,
        'Waktu (s)': t_elapsed
    })

    print(f"    □ Selesai | Accuracy: {acc:.4f} | F1: {f1:.4f} | "
          f"AUC: {auc:.4f} | Waktu: {t_elapsed:.2f}s")

# =====
# TABEL PERBANDINGAN HASIL EVALUASI
# =====

df_results = pd.DataFrame(results)

print("\n" + "=" * 65)
print("  TABEL PERBANDINGAN HASIL EVALUASI SEMUA METODE")
print("=" * 65)
display(df_results.set_index('Metode'))

best_f1 = df_results.loc[df_results['F1-Score'].idxmax(), 'Metode']
best_auc = df_results.loc[df_results['AUC-ROC'].idxmax(), 'Metode']
print(f"\n Metode terbaik (F1-Score) : {best_f1}")
print(f" Metode terbaik (AUC-ROC)   : {best_auc}")

# =====
# VISUALISASI: BAR CHART PERBANDINGAN
# =====

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle('Perbandingan Performa 5 Skenario Imputasi (Random
Forest)',
             fontsize=14, fontweight='bold')

colors = ['#7f8c8d', '#3498db', '#2ecc71', '#9b59b6', '#e67e22']
methods = df_results['Metode'].tolist()

for ax, metric, ylim in zip(axes,
                             ['F1-Score', 'AUC-ROC', 'Waktu (s)'],
                             [(0.80, 1.0), (0.80, 1.0), None]):
    bars = ax.bar(methods, df_results[metric], color=colors,
                  edgecolor='black', linewidth=0.8)
    ax.set_title(metric, fontsize=13, fontweight='bold')
    ax.set_ylabel(metric, fontsize=11)
    if ylim:
        ax.set_ylim(*ylim)
    for bar, val in zip(bars, df_results[metric]):
        label = f'{val:.4f}' if metric != 'Waktu (s)' else
f'{val:.1f}s'

```

```

        offset = 0.001 if metric != 'Waktu (s)' else 0.1
        ax.text(bar.get_x() + bar.get_width() / 2.,
                bar.get_height() + offset,
                label, ha='center', va='bottom', fontsize=9,
fontweight='bold')

plt.tight_layout()
plt.savefig('fig5_comparison.png', bbox_inches='tight', dpi=100)
plt.show()

# =====
# HEATMAP SEMUA METRIK
# =====

metric_cols = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'AUC-
ROC']
fig, ax = plt.subplots(figsize=(10, 4))
sns.heatmap(df_results.set_index('Metode')[metric_cols],
            annot=True, fmt='.4f', cmap='YlOrRd',
            linewidths=0.5, ax=ax, cbar_kws={'label': 'Nilai Metrik'})
ax.set_title('Heatmap Perbandingan Semua Metrik Evaluasi',
            fontsize=13, fontweight='bold')
ax.set_xlabel('Metrik', fontsize=11)
ax.set_ylabel('Metode Imputasi', fontsize=11)
plt.tight_layout()
plt.savefig('fig6_heatmap_metrics.png', bbox_inches='tight', dpi=100)
plt.show()

print("☑ Semua visualisasi berhasil dibuat.")

```

=====

TRAINING RANDOM FOREST CLASSIFIER (5 SKENARIO)

=====

```

[1/5] Training dengan metode: Baseline...
☑ Selesai | Accuracy: 0.8625 | F1: 0.8530 | AUC: 0.8979 | Waktu:
17.16s

[2/5] Training dengan metode: Mean...
☑ Selesai | Accuracy: 0.8569 | F1: 0.8459 | AUC: 0.8884 | Waktu:
30.80s

[3/5] Training dengan metode: Median...
☑ Selesai | Accuracy: 0.8571 | F1: 0.8463 | AUC: 0.8882 | Waktu:
30.90s

[4/5] Training dengan metode: KNN...
☑ Selesai | Accuracy: 0.8560 | F1: 0.8448 | AUC: 0.8847 | Waktu:
34.72s

```

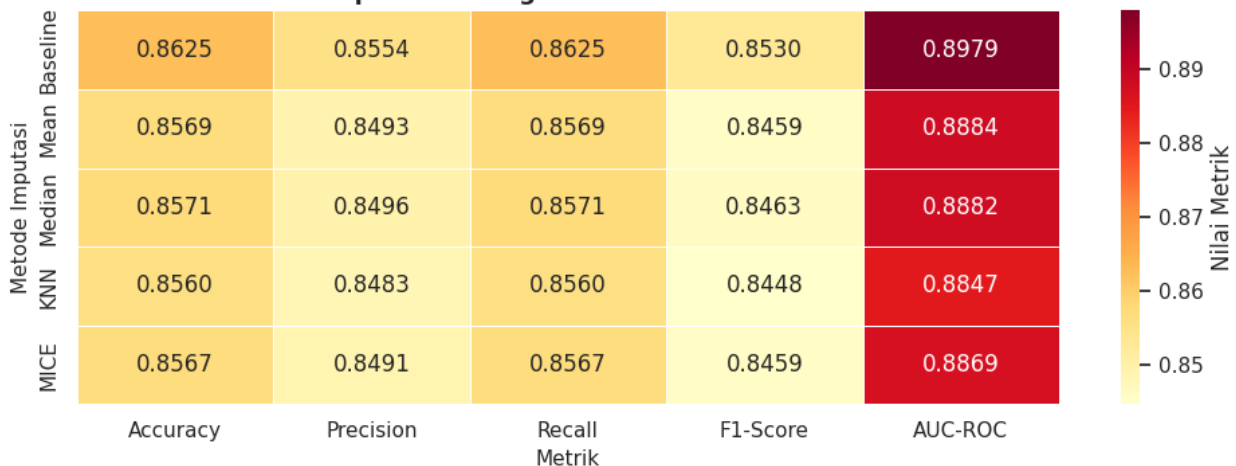
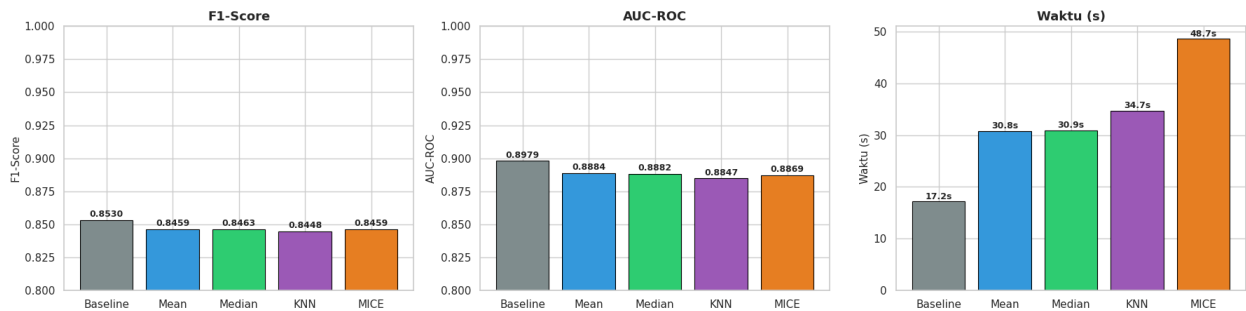
```
[5/5] Training dengan metode: MICE...
```

Selesai | Accuracy: 0.8567 | F1: 0.8459 | AUC: 0.8869 | Waktu: 48.65s

TABEL PERBANDINGAN HASIL EVALUASI SEMUA METODE

[illegible]

```
Metode terbaik (F1-Score) : Baseline
Metode terbaik (AUC-ROC) : Baseline
```



□ Semua visualisasi berhasil dibuat.